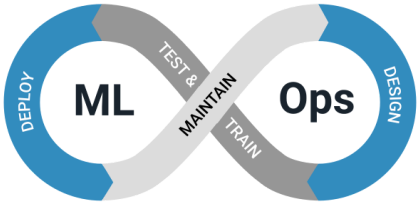
DSAI 406

Sheet 1



Scenario: Standardizing the ML Development and Deployment Environment

You are an **MLOps Engineer** at a company developing several machine learning models for different business applications. Recently, the ML team has faced multiple issues when sharing and deploying their models.

Some of the problems reported include:

- The training code works on one developer's laptop but fails on another machine.
- Different team members use **different Python versions and library versions**.
- Models trained locally behave differently when deployed on the cloud server.
- Setting up the environment for new team members takes several hours.

The current project uses the following Python libraries:

- numpy
- pandas
- scikit-learn
- matplotlib
- mlflow

To address these issues, your goal is to create a **reproducible development and deployment environment** that can run consistently across developer machines, testing servers, and production environments.

Your proposed solution includes:

- Creating **Python virtual environments** for development
- Maintaining a **requirements.txt file** for dependency management
- Using **Docker** to build standardized environments for training and deployment

Below is a simplified training script used by the team:

```
import numpy as np
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

model = RandomForestClassifier(n_estimators=100)
model.fit(X, y)

print("Training completed successfully")
```

However, the environment required to run this script is not clearly documented or standardized.

Questions

1. Environment Isolation Using Virtual Environments

Explain how you would use **Python virtual environments** to isolate project dependencies.

Your answer should include:

- Steps to create a virtual environment
- How to activate the environment
- How to install required dependencies

Explain why virtual environments are important for **reproducibility and avoiding dependency conflicts**.

---

2. Dependency Management Using requirements.txt

Describe how you would create and maintain a **requirements.txt** file for the project.

Explain:

- How dependencies can be exported into a requirements file
- How other developers can use this file to recreate the same environment
- Why version pinning (e.g., `numpy==1.26.0`) is important in ML projects

---

3. Containerization Using Docker

To ensure the application runs consistently across all environments, you decide to containerize the project using **Docker**.

Explain how Docker helps solve problems related to:

- Environment inconsistency
- Dependency conflicts
- Deployment portability

Describe the key components of a **Dockerfile**, including:

- Selecting a base image
- Installing dependencies
- Copying project code
- Running the training or inference script

---

4. Docker Image and Container Workflow

Explain the difference between a **Docker image** and a **Docker container**.

Then describe the typical workflow for:

1. Building a Docker image from a Dockerfile
2. Running a container from the image
3. Testing the application inside the container

---

5. Benefits for MLOps and Collaboration

Explain how combining **virtual environments, requirements files, and Docker containers** improves the overall MLOps

workflow.

Discuss how this approach benefits:

- Team collaboration
- Reproducibility of experiments
- Deployment to cloud or production environments

Scenario: Negotiating ML Resource Allocation Between Teams

You are an **MLOps Engineer** at a technology company that develops multiple machine learning products. Your organization has **a shared GPU cluster with 16 GPUs**, used by several teams for different ML workloads.

Recently, conflicts have emerged between the **Data Science team** and the **ML Engineering team** regarding GPU usage. Both teams claim their workloads are critical and require more GPU resources. As a result, jobs frequently fail or remain in queue due to insufficient GPU availability.

Your task is to design a **fair and efficient GPU allocation strategy** that balances technical requirements and business priorities while reducing team conflicts.

The main workloads currently running on the cluster are:

Workload	Description
Real-time fraud detection	ML model serving that must respond instantly to financial transactions
Daily model training	Training production models used in recommendation systems
Weekly model retraining	Periodic retraining using updated datasets
Hyperparameter tuning	Running multiple experiments to improve model performance
Exploratory research	Data scientists experimenting with new model ideas

The GPU cluster currently has **16 GPUs**, but demand often exceeds supply.

Below is a simplified representation of workloads submitted to the cluster:

```
workloads = [
  {"name": "Fraud Detection", "priority": "High"},
  {"name": "Daily Training", "priority": "High"},
  {"name": "Weekly Retraining", "priority": "Medium"},
  {"name": "Hyperparameter Tuning", "priority": "Medium"},
  {"name": "Exploratory Research", "priority": "Low"},
]

gpu_pool = {"total": 16, "allocated": 0}
```

Your goal is to create a strategy that allocates GPUs efficiently while maintaining fairness across teams.

Questions

1. Workload Prioritization

Explain how you would classify workloads into **High, Medium, and Low priority categories**.

- Describe the criteria used for prioritization (e.g., business impact, service-level agreements, latency requirements).
- Provide examples of which workloads should receive **dedicated GPUs** versus **shared GPUs**.

---

2. Resource Allocation Strategy

Design a **GPU allocation framework** that ensures critical workloads always have enough resources.

Your answer should include:

- Dedicated GPUs for high-priority workloads
- Shared GPU pools for medium-priority workloads
- Opportunistic or preemptible GPUs for low-priority workloads

Explain how this strategy improves cluster utilization and reduces resource conflicts.

---

3. Monitoring and Cost Attribution

To reduce disputes between teams, management wants to **track GPU usage per team and per project**.

Explain how you would implement a monitoring system that:

- Logs GPU usage by team and workload
- Tracks training job duration and GPU consumption
- Produces dashboards showing resource usage

Mention any tools or approaches that could help achieve this.

---

4. Dynamic Scheduling and Fairness

Sometimes a high-priority workload may require additional GPUs urgently.

Explain how **dynamic scheduling or preemption** could allow high-priority jobs to temporarily take GPUs from lower-priority workloads.

Discuss potential risks and how you would minimize disruptions to other teams.

---

5. Collaboration and Conflict Resolution

In addition to technical solutions, explain how you would handle the **organizational conflict between teams**.

Discuss strategies such as:

- Transparent GPU usage dashboards
- Clear resource allocation policies
- Regular review meetings between teams

Explain how these practices help maintain fairness and prevent future conflicts.

Scenario: Resolving a Model Versioning Nightmare

You are an **MLOps Engineer** at a company that deploys multiple machine learning models in production. These models power critical services such as recommendation systems, fraud detection, and customer analytics.

Recently, a major issue occurred: the production system began producing incorrect predictions, affecting business operations. When the team attempted to investigate the issue, they discovered several serious problems:

- No one knows **which model version is currently running in production**.
- Multiple developers have trained models locally and manually deployed them.
- Some model files were **overwritten with newer versions**, making it impossible to reproduce previous results.
- There is **no record of which dataset or hyperparameters were used** for different model versions.

Management is concerned that similar issues could lead to **significant financial losses** or regulatory compliance problems.

Your task is to **design a robust model versioning and deployment strategy** that ensures reproducibility, traceability, and safe deployment of machine learning models.

Below is a simplified example of how one developer currently logs a model using MLflow:

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
model = RandomForestClassifier(n_estimators=100)

with mlflow.start_run() as run:
  model.fit(X, y)

  mlflow.sklearn.log_model(
     model,
     artifact_path="iris_model",
     registered_model_name="IrisRFClassifier"
  )

  print(f"Model logged in run ID: {run.info.run_id}")
```

However, the organization currently lacks a **clear workflow for versioning, approving, and deploying models**.

Questions

1. Model Versioning and Metadata Tracking

Explain how you would use **MLflow or a similar metadata store** to manage model versions.

Your answer should include:

- Logging training parameters and evaluation metrics
- Tracking datasets and feature versions used during training
- Registering models in a **central model registry**

Explain why **metadata tracking is critical for reproducibility and debugging**.

---

2. Immutable Model Artifacts

One major problem is that developers sometimes **overwrite existing model files**.

Explain why model artifacts should be **immutable**.

Describe a strategy to ensure that:

- Each trained model is stored as a **unique version**
- Previous versions remain accessible for debugging or rollback
- Model artifacts are linked to the corresponding experiment run

---

3. CI/CD Pipeline for ML Models

Design a **CI/CD pipeline for machine learning models** that ensures controlled releases.

Your pipeline should include the following stages:

- **Build:** Train and validate the model

- **Register:** Store the model in the model registry
- **Approve:** Require manual or automated approval before deployment
- **Deploy:** Release the approved model to production
- **Monitor:** Track performance and detect issues

Explain how this pipeline improves reliability and reduces deployment risks.

---

4. Deployment Monitoring and Rollback Strategy

Suppose a newly deployed model causes a sudden drop in prediction accuracy.

Explain how you would implement a **rollback strategy**.

Your answer should include:

- Monitoring model performance in production
- Detecting anomalies or performance degradation
- Automatically reverting to a previous stable model version

Discuss how proper model versioning simplifies this process.

---

5. Auditability and Governance

For compliance and accountability, management wants a system where every deployed model has a clear history.

Explain how your system would ensure that each model deployment records:

- The **model version**
- The **developer or team responsible**
- The **training dataset version**
- The **deployment timestamp**

Discuss how these audit trails help with **debugging, compliance, and accountability**.